

Reviewer Vigilance Tracker — Build Guide — Absolute Beginner Edition

Project: A bot that measures whether your human code reviewers still really check pull requests after an AI has reviewed them first — deployed free on GitHub Actions.

Who this is for: Someone who has never built or deployed a Python project before. Every term is explained, every command is spelled out. You will build on fake data first and only point it at a real repo once the logic is proven.

Before you start: 15 words you need to know

- **Pull request (PR)** — a proposed change to code. Someone writes it; someone else reviews and approves it before it merges.
- **Reviewer** — the human who reads a PR and clicks Approve (or asks for changes). This tool is about them.
- **AI review bot** — a tool like CodeRabbit or GitHub Copilot that auto-reviews a PR and leaves a "looks good" before the human arrives.
- **Automation complacency / rubber stamp** — when the human trusts the AI's green check and approves without really reading. The exact thing we measure.
- **Vigilance score** — a 0–100 number (higher = more engaged) we compute per reviewer, per period, from PR data.
- **Python** — the programming language the tool is written in.
- **Terminal (command line)** — the text window where you type commands. Windows: *PowerShell*. Mac: *Terminal*.
- **Virtual environment (venv)** — a private bubble of Python for this project, so its libraries don't tangle with the rest of your computer.
- **Package / pip** — [pip](#) installs pre-written code (packages) your project builds on.
- **Repository (repo) / GitHub** — a saved, versioned copy of your code on GitHub.com.
- **GitHub Actions / workflow** — free automation that runs your code *inside* GitHub on a schedule or when something happens. This is your "server" — you don't rent one.
- **GITHUB_TOKEN** — a temporary password GitHub gives your workflow so it can post issues/comments. You never create it by hand.
- **Personal access token (PAT)** — a password-for-programs you generate to read data from GitHub when running on your own laptop.
- **Unit test / pytest** — small automated checks that prove your logic works. [pytest](#) runs them.
- **Secret** — a sensitive value (like a Slack URL) stored safely in GitHub, never written in code.

One rule for the whole project: the scoring logic is built and tested on *simulated* data first. You do not point it at a real repository until Step 7, and you do not trust its numbers as truth until Step 11.

STEP 1 — Set up your computer (2–3 hours, one-time)

1.1. Install Python: go to [python.org](#) → Downloads → install the latest 3.x version.

- **Windows, critical:** on the first installer screen, tick "**Add python.exe to PATH**" before clicking Install. Miss it and the terminal won't find Python.
- **Mac:** just run the installer.

1.2. Install Git from [git-scm.com](#) (accept all defaults). Git is what saves your code to GitHub.

1.3. Install VS Code from code.visualstudio.com — the free editor where you'll read and write code.

1.4. Open a terminal: Windows — press the Windows key, type [powershell](#), Enter. Mac — press Cmd+Space, type [terminal](#), Enter.

1.5. Check the installs worked. Type each line, pressing Enter after each:

```
python --version
git --version
```

Each should print a version number (e.g. [Python 3.12.4](#)). If you get "not recognized", Python wasn't added to PATH — re-run the installer and tick the box.

CHECKPOINT: [python --version](#) and [git --version](#) both print numbers, and VS Code opens.

STEP 2 — Create the project and get the code (30 minutes)

You have two paths. **Path A (recommended for beginners):** use the ready-made package that accompanies this guide — you'll read and run real, working code. **Path B:** type each file yourself as the steps describe. Either way, start here:

2.1. In the terminal, make a project folder and enter it ([mkdir](#) makes a folder, [cd](#) enters it):

```
mkdir vigilance-tool
cd vigilance-tool
```

2.2. If using Path A, unzip the provided [vigilance-tool.zip](#) into this folder now (so you see [vigilance/](#), [tests/](#), [requirements.txt](#) inside).

2.3. Create a **virtual environment** (the private bubble) and turn it on:

```
# create it
python -m venv .venv

# turn it on — Windows PowerShell:
.venv\Scripts\activate
# turn it on — Mac:
source .venv/bin/activate
```

Your prompt now shows ([.venv](#)) at the start. That means you're inside the bubble. Do this every time you work on the project.

2.4. Install the libraries the tool needs:

```
pip install requests pytest
```

[requests](#) talks to GitHub's API; [pytest](#) runs your tests. (The full package's [requirements.txt](#) lists the same.)

2.5. Open the project in VS Code — type [code .](#) in the terminal (note the dot).

CHECKPOINT: your terminal shows ([.venv](#)); [pip list](#) shows [requests](#); VS Code is showing your [vigilance-tool](#) folder.

STEP 3 — The data contract: `models.py` (half a day)

What you're building: the shapes of data every other part of the tool passes around. Building this *first* is the trick that keeps everything else simple: if the real-GitHub reader and the fake-data simulator both produce the same shapes, the scoring code can't tell them apart — so you can test it without touching the internet.

3.1. Create `vigilance/models.py` defining three simple shapes (dataclasses):

- **ReviewEvent** — one review on a PR: who, is-it-a-bot, state (APPROVED/...), timestamp, how many inline comments.
- **PRMeta** — the PR's size: lines added, lines deleted, files changed. Used to judge engagement fairly (a 5-line PR needs fewer comments than a 500-line one).
- **ScoredReview** — one human review after scoring: was it a rubber stamp, comment density, minutes after the bot.

Think of these as the standard envelopes every module passes around. In the package this is already written — open it and read it.

CHECKPOINT: `models.py` exists and `python -c "import vigilance.models"` runs with no error.

STEP 4 — The scoring engine: `scoring.py` (1–2 days)

What you're building: the heart of the tool — a *pure function* (data in, numbers out, no internet) that turns reviews into a vigilance score. Because it's pure, it's easy to test and impossible for a network glitch to break.

4.1. Define a rubber stamp precisely. A review is a rubber stamp when **all** of these are true:

- a human **APPROVED** the PR,
- they left **zero** comments,
- an **AI bot had already reviewed** the same PR first, and
- they approved within **15 minutes** of that bot review.

4.2. The score (0–100, higher = more engaged), per rolling group of reviews:

```
vigilance_score = 100
- (rubber_stamp_rate * 60)
- (low_comment_penalty * 40)
```

where `low_comment_penalty` rises as comment density (comments per 100 changed lines) falls below a target. Every number here lives in one `ScoringConfig` object, so you tune behaviour without touching logic.

4.3. Compute it in **rolling windows** (e.g. every 5 reviews) and add `flag_declining_reviewers()` — flag someone only when their score *falls* by 20+ points from their first window to their latest. **The trend is the signal, not one low score.**

CHECKPOINT: `scoring.py` imports cleanly and exposes `score_reviews`, `compute_vigilance_scores`, and `flag_declining_reviewers`.

STEP 5 — Prove it with a simulator: `simulate.py` (half a day)

What you're building: fake but realistic data where *you already know the answer*. That's what lets you trust the scorer before real data ever enters.

5.1. Create `simulate.py` with `simulate_drift()` modelling two reviewers over 40 weeks:

- **alice** — starts careful, gradually rubber-stamps faster with fewer comments (the drift).
- **bob** — stays careful the whole time (the control, to prove the tool doesn't just flag everyone).

5.2. Run the whole pipeline on the fake data:

```
python -m vigilance report --source simulate
```

You should see alice's score fall off a cliff (about 82 → 13) and get flagged, while bob holds in the 80s–90s and is left alone.

CHECKPOINT: the report prints, alice is flagged, bob is not. The tool catches the pattern it's supposed to and ignores the reviewer who's fine.

STEP 6 — Lock it with tests: `pytest` (half a day)

What you're building: automated checks so you can change code later without fear. Because the simulator knows the truth, the tests assert the scorer reaches it.

6.1. The tests (in `tests/`) assert the properties that matter: the drifter gets flagged, the steady reviewer does not, a fast zero-comment approval is a rubber stamp *only if a bot reviewed first*.

6.2. Run them:

```
pytest -q
```

You want to see all tests pass (the package ships 13 passing tests).

6.3. If something fails, match the message:

Message contains	What it means	Fix
<code>ModuleNotFoundError</code>	the venv bubble is off, or you're in the wrong folder	re-run the activate command from 2.3, from the project root
<code>ImportError: cannot import name...</code>	a function name doesn't match	check spelling against <code>scoring.py</code>
<code>assert ... alice not flagged</code>	a threshold in <code>ScoringConfig</code> is off	compare your numbers to Step 4.2

CHECKPOINT: `pytest -q` shows all green. The hard part is now done — everything after this is plumbing.

STEP 7 — Point it at a real repo: `github_fetcher.py` (1–2 days)

What you're building: the reader that pulls real PR + review history from GitHub and turns it into the same `ReviewEvent` shapes — so the tested scorer works unchanged.

7.1. Get a token (a password-for-programs): on GitHub, go to **Settings** → **Developer settings** → **Personal access tokens** → **Fine-grained tokens** → **Generate**. For public repos you need no special permissions. Copy it somewhere safe (a password manager).

7.2. Put the token in your terminal so code can read it (don't paste it into any file):

```
# Windows PowerShell:
$env:GITHUB_TOKEN="ghp_your_token_here"
# Mac:
export GITHUB_TOKEN="ghp_your_token_here"
```

7.3. Run against a real public repo (pick any that uses PR reviews):

```
python -m vigilance report --source github --owner pallets --repo flask --max-prs 40
```

7.4. If it fails, match the error:

Error contains	What it means	Fix
401 / Bad credentials	wrong or expired token	regenerate the token, re-set it (7.2), no spaces
rate limit / 403	too many requests without a token	make sure the token is set; results are cached so re-runs are free
Not Found	owner/repo misspelled, or a private repo your token can't see	check spelling; use a token with access

The score only makes sense if the tool correctly spots which reviews were the AI bot's. If your team uses a bot not in the list, add its login to `AI_BOT_LOGINS` in `github_fetcher.py`.

CHECKPOINT: a real repo prints real reviewers and scores, using the exact scorer your tests already proved.

STEP 8 — The outputs: console, CSV, dashboard (2–3 days)

What you're building: three ways to see the result — a terminal table, a CSV for Excel, and a shareable HTML dashboard with trend lines.

8.1. Generate all three at once:

```
python -m vigilance report --source simulate --html --csv
```

8.2. Open `output/vigilance.html` in your browser. You'll see per-reviewer trend lines (a red falling line for the drifter, a green steady line for the control), a flagged panel, and a data table. `output/vigilance.csv` opens in Excel/Sheets.

CHECKPOINT: the dashboard opens and shows the drifting reviewer's line dropping while the steady one holds flat.

STEP 9 — The bot brain: alerts & the friction role (2–3 days)

What you're building: the parts that *act*, not just measure — an alert when someone's trend drops, and the "friction role" that helps prevent complacency in the first place.

9.1. Alerts (`alerting.py`): send a Slack message (and/or email) when a reviewer is flagged. Configured with environment variables so no secret ever lives in code. Messaging is deliberately gentle — "worth a look," never "this person is failing."

9.2. Friction role ([friction.py](#)): on high-risk PRs (big diffs, many files, or sensitive paths like [auth/payment](#)), *withhold the AI's verdict* until the human commits their own — then reveal both. Try the demo:

```
python -m vigilance friction-demo
```

The **CAUGHT** line — human flags what the AI approved — is the exact outcome the whole tool exists to protect.

CHECKPOINT: the friction demo prints WITHHOLD/show decisions per PR and the reconciliation outcomes.

STEP 10 — Deploy it live on GitHub Actions (half a day)

What you're building: the bot goes live — running inside GitHub for free, with no server to rent. Two workflows do the work; you just flip a few switches.

10.1. Put the tool in a GitHub repo. Create a free account at [github.com](#), click **New repository**, name it [vigilance-tool](#), set it **Private**, then in your terminal:

```
git init
git add vigilance requirements.txt .github tests
git commit -m "vigilance bot"
```

(GitHub then shows you two lines — `git remote add ...` and `git push ...` — copy those from its screen and run them.)

10.2. Turn on Pages (for the dashboard): repo **Settings** → **Pages** → **Source: GitHub Actions**. Your dashboard will live at <https://<you>.github.io/vigilance-tool/>.

10.3. Allow writing: repo **Settings** → **Actions** → **General** → **Workflow permissions** → **Read and write** → Save. This lets the bot post the digest issue and PR nudges.

10.4. (Optional) Slack: make an Incoming Webhook, then repo **Settings** → **Secrets and variables** → **Actions** → **New repository secret**, name it [SLACK_WEBHOOK_URL](#).

10.5. Run it once: repo **Actions** tab → **Reviewer Vigilance — Weekly Digest** → **Run workflow**. Within a minute you get a digest issue, a live dashboard URL, and (if set) a Slack ping. The PR-nudge workflow then runs by itself whenever someone approves a PR.

CHECKPOINT: the weekly workflow runs green, the digest issue appears, and the dashboard is live at your Pages URL.

STEP 11 — The trial run: validate before you trust it (4–8 weeks — never skip)

What you're building: confidence that the number means something. Right now the score is a *plausible* proxy, not proven truth. This step is the equivalent of test-flying before you carry passengers.

11.1. Let the bot run on a real repo for several weeks. Read the weekly digests. Don't change thresholds mid-run — write down ideas for later.

11.2. Correlate the score against reality: do reviewers with falling scores actually ship more escaped bugs / hotfixes / reverts? Until you've seen that link on your own team, treat the score as a conversation-starter, not a verdict.

11.3. Roll it out as a team smoke detector, not a scorecard. Never wire these numbers into individual performance reviews — that just trains people to game the metric with padded comments and artificial delays. Use it to redesign the workflow: rotate reviewers, add friction on the risky PRs.

CHECKPOINT: you can say, from your own data, whether a low vigilance score really predicts more missed defects — and the team sees the tool as a shared safety net, not surveillance.

STEP 12 — Keep it healthy (ongoing habits)

- Add any new AI review bot your team adopts to [AI_BOT_LOGINS](#) — the whole signal depends on identifying the bot's reviews.
- Re-check thresholds quarterly against how your team actually reviews.
- Consider adding a review-latency signal (time-vs-PR-size) as a fourth input — the data is already fetched.
- Rotate your personal access token periodically; the workflow's [GITHUB_TOKEN](#) rotates itself.
- Keep the caveat visible: a fast, low-comment approval isn't always complacency — sometimes the PR really was trivial. The tool flags a *trend*; it stays a heuristic.

Realistic timeline

Steps	Calendar time
1–2 (computer + project setup)	one afternoon
3–4 (models + scoring engine)	2–3 days
5–6 (simulator + tests)	1–2 days
7–8 (real GitHub + outputs)	3–5 days
9–10 (bot brain + deploy)	3–5 days
11 (validation trial)	4–8 weeks

Total: about 2–3 weeks of building, then a multi-week trial. The slowest step — the validation trial — is the one that decides whether the tool is trustworthy. Every "our metric backfired" story skipped it.

This tool produces a heuristic, not a certified measurement of anyone's ability. Use it as a team early-warning system to target attention and improve the review workflow — never as an individual performance metric. A fast approval is not proof of negligence; the signal is a sustained downward trend, read alongside human judgment.